OAP200V

# Advanced inheritance

## Abstract classes vs interfaces

Lecturer: Boban Vesin

# Abstract class vs interface

- Both are used to achieve abstraction
- Abstraction allows you to define methods that must be implemented by derived classes or implementing classes, without providing the actual implementation in the abstract class or interface itself
- However, there are important differences between abstract classes and interfaces, which affect how they are used and the level of abstraction they provide

# Abstract class

- Class that cannot be instantiated on its own and is meant to be subclassed.
- It can contain both:
  - abstract methods (methods without an implementation) and
  - concrete methods (methods with an implementation).

# Key features of Abstract class

- **Abstract Methods**: These methods are declared without an implementation. Subclasses must implement these methods.
- **Concrete Methods**: An abstract class can have concrete methods with implementations. This allows subclasses to inherit common behavior.
- **Fields**: Abstract classes can have fields (variables) and constructors. These can be used to maintain the state and to initialize object state when an abstract class is subclassed.
- **Multiple Inheritance Limitation**: A class in Java can extend only one abstract class due to Java's single inheritance model.

# Example

```
abstract class Animal {
    String name;

    Animal(String name) {
        this.name = name;
    }

    abstract void makeSound();

    void sleep() {
        System.out.println(name + " is sleeping.");
    }
}

class Dog extends Animal {
    Dog(String name) {
        super(name);
    }

    void makeSound() {
        System.out.println("Woof!");
    }
}
```

# Interfaces

- It is a reference type, similar to a class, that can contain only
  - Constants,
  - method signatures,
  - default methods,
  - static methods, and
  - nested types.
- Interfaces cannot contain instance fields and constructors.
- A class that implements an interface must provide an implementation for all of the methods declared in the interface.

# Key features

- **Abstract Methods**: By default, all methods in an interface are abstract (unless they are static or default methods).
- **Default Methods**: Java 8 introduced default methods in interfaces, which allow methods with implementations. This enables interfaces to evolve without breaking the implementing classes.
- **Static Methods**: Interfaces can also have static methods with implementations.
- **No State**: Interfaces cannot have instance fields; they cannot maintain state.
- **Multiple Inheritance**: A class can implement multiple interfaces, which allows for a form of multiple inheritance, enabling a class to inherit behaviors from multiple sources.

# Example

```
interface Animal {
    void makeSound();

    default void sleep() {
        System.out.println("Animal is sleeping.");
    }
}

class Dog implements Animal {
    public void makeSound() {
        System.out.println("Woof!");
    }
}
```

# Differences Between Abstract Classes and Interfaces

| Feature | Abstract Class | Interface |
|---|---|---|
| **Method Implementation** | Can have both abstract and concrete methods. | Can have abstract methods, and since Java 8, can also have default and static methods. |
| **Fields** | Can have instance fields (variables). | Cannot have instance fields, only constants (`public static final` fields). |
| **Constructors** | Can have constructors. | Cannot have constructors. |
| **Inheritance** | A class can extend only one abstract class (single inheritance). | A class can implement multiple interfaces (multiple inheritance). |
| **Use Case** | Use when you want to share code among closely related classes. | Use to define a contract that unrelated classes can implement. |

# When to Use What?

- **Abstract Class**: Use an abstract class when you want to share common code among several closely related classes or when you want to define a base class with some default behavior and some mandatory methods that subclasses must implement.
- **Interface**: Use an interface when you want to define a contract for classes that may not be related, or when you need to provide a way to achieve multiple inheritance (since Java allows multiple interfaces to be implemented by a single class). Interfaces are also useful for defining APIs and ensuring that different classes adhere to a certain protocol.

# Questions?